



React Hook Form

1. What is React Hook Form?

React Hook Form is a popular React library used to create and manage forms easily.

Normally, in React forms, we may need to manage:

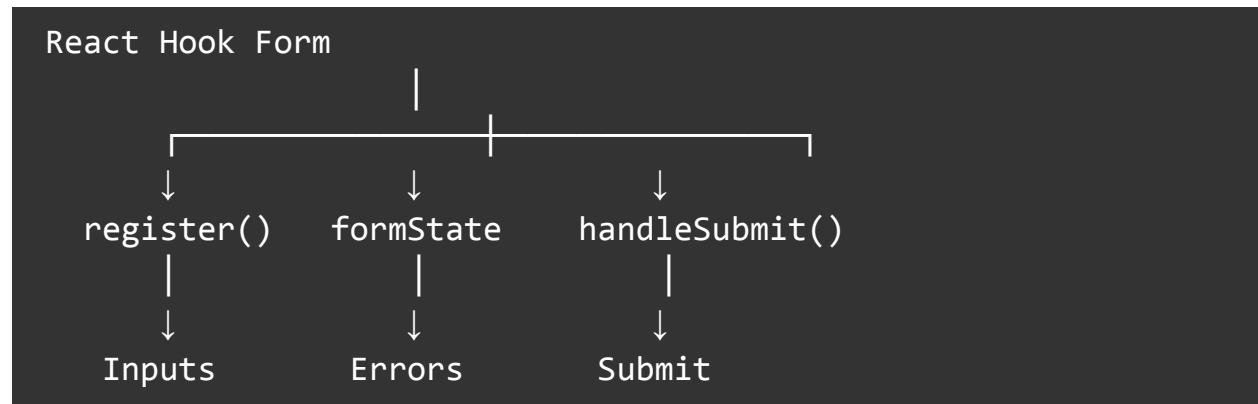
- Input values
- Form submission
- Validation
- Error messages
- Form state
- Change events

React Hook Form simplifies all of these.

Without React Hook Form

```
User Input
  ↓
useState()
  ↓
onChange()
  ↓
Store Value
  ↓
Validate
  ↓
Show Error
  ↓
Submit Form
```

With React Hook Form



2. Installation

First install React Hook Form:

```
npm install react-hook-form
```

Then import `useForm`:

```
import { useForm } from "react-hook-form";
```

3. useForm()

`useForm()` is the main hook provided by React Hook Form.

It gives us several important functions and objects.

Syntax

```
const {  
  register,  
  handleSubmit,  
  formState  
} = useForm();
```

Visual Understanding

```
useForm()  
├── register()  
│   └── → Connect input with React Hook Form  
├── handleSubmit()  
│   └── → Handle form submission  
└── formState  
    └── → Information about form  
        ├── errors  
        ├── isSubmitting  
        ├── isSubmitted  
        └── isValid
```

The four important concepts we will learn:

1. `register()`
 2. `formState`
 3. Validation
 4. `handleSubmit()`
-

4. register()

What is register()?

`register()` connects an HTML input with React Hook Form.

It tells React Hook Form:

"I want you to manage this input."

Syntax

```
<input {...register("username")} />
```

Here:

```
register("username")
  |
  └─ Field name
```

The field will be stored using the name `username`.

Example

```
import { useForm } from "react-hook-form";

function App() {

  const { register } = useForm();

  return (
    <form>

      <input
        {...register("username")}
      />
```

```
    </form>
  );
}
```

What happens?

When we write:

```
{...register("username")}
```

React Hook Form connects the input to its internal form management.

Conceptually:

```
<input>
  |
  | register()
  ↓
React Hook Form
  |
  ↓
username
  |
  ↓
Form Data
```

5. Why do we use {...register()}?

You will usually see:

```
{...register("email")}
```

The `...` is the **spread operator**.

`register()` returns properties and event handlers required by React Hook Form.

Conceptually:

```
register("email")
```

returns something similar to:

```
{
  name: "email",
  onChange: ...,
  onBlur: ...,
  ref: ...
}
```

Therefore:

```
<input {...register("email")} />
```

connects all those properties to the input.

6. formState

What is formState?

`formState` contains information about the current state of the form.

For example:

```
const {  
  formState  
} = useForm();
```

We can access:

`formState.errors`

to get validation errors.

Important formState properties

Some commonly used properties are:

```
formState  
├── errors  
│   └── Validation errors  
├── isSubmitting  
│   └── Form is currently submitting  
├── isSubmitted  
│   └── Form has been submitted  
└── isSubmitSuccessful  
    └── Submission succeeded
```

```
|  
└─ isValid  
    → Form is valid
```

For beginner-level forms, the most important one is:

`formState.errors`

7. Validation

React Hook Form allows us to add validation rules directly inside `register()`.

Example

```
<input  
  {...register("username", {  
    required: true  
    Message:"username is required"  
  })}  
>
```

Here: `required: true`

means: Username cannot be empty.

Common validation rules

Required

```
register("username", {  
  required: true,  
})
```


Minimum length

```
register("username", {  
  minLength: 3  
  message: "----"  
})
```

Maximum length

```
register("username", {  
  maxLength: 20  
})
```

Minimum number

```
register("age", {  
  min: 18  
})
```

Maximum number

```
register("age", {  
  max: 60  
})
```

Pattern

```
register("email", {  
  pattern: /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/,  
  Message: "----"  
})
```

8. Validation with Error Message

We can provide an error message using the `message` property.

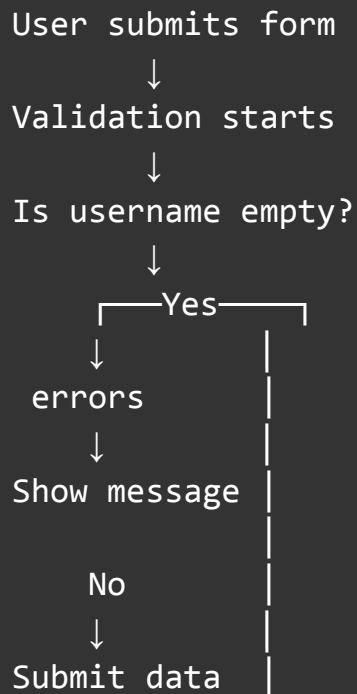
Syntax

```
register("username", {
  required: "Username is required"
})
```

Then:

```
{errors.username && (
  <p>{errors.username.message}</p>
)}
```

Flow



9. handleSubmit()

What is handleSubmit()?

`handleSubmit()` handles form submission.

It performs validation first.

If validation succeeds, it calls our submit function.

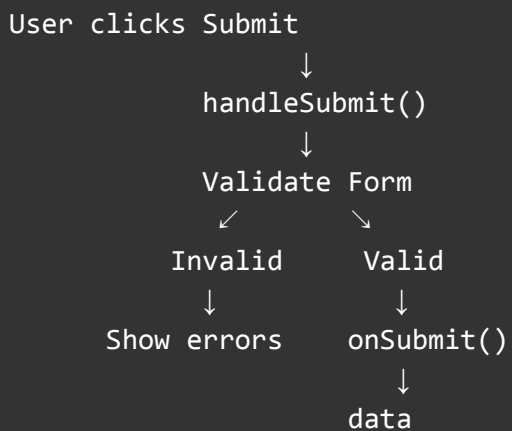
Syntax

```
<form onSubmit={handleSubmit(onSubmit)}>
```

Where:

```
const onSubmit = (data) => {  
  console.log(data);  
};
```

Complete flow



10. Example 1 — Simple Login Form

Let's create a login form using:

```
register()  
formState  
Validation  
handleSubmit()
```

Step 1: Import useForm

```
import { useForm } from "react-hook-form";
```

Step 2: Create useForm()

```
const {  
  register,  
  handleSubmit,  
  formState: { errors }  
} = useForm();
```

Here we are extracting:

```
register()  
  ↓  
Connect inputs  
  
handleSubmit()  
  ↓  
Handle form submission  
  
errors  
  ↓  
Store validation errors
```

Step 3: Create submit function

```
const onSubmit = (data) => {  
  console.log(data);  
};
```

`data` contains the submitted form values.

Step 4: Create input

```
<input  
  {...register("email", {  
    required: "Email is required"  
  })}  
>
```

Step 5: Display error

```
{errors.email && (  
  <p>{errors.email.message}</p>  
)}
```

Complete Example 1

```
import { useForm } from "react-hook-form";

function LoginForm() {

  const {
    register,
    handleSubmit,
    formState: { errors }
  } = useForm();

  const onSubmit = (data) => {
    console.log(data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>

      <div>
        <label>Email</label>

        <input
          type="email"
          {...register("email", {
            required: "Email is required"
          })}
        />

        {errors.email && (
          <p>{errors.email.message}</p>
        )}
      </div>

      <div>
        <label>Password</label>

        <input
          type="password"
          {...register("password", {
            required: "Password is required",
            minLength: {
              value: 6,
```

```

        message: "Password must be at least 6 characters"
      }
    }
  }
  />

  {errors.password && (
    <p>{errors.password.message}</p>
  )}
</div>

<button type="submit">
  Login
</button>

</form>
);
}

export default LoginForm;

```

11. Example 1 — Explanation

Step 1

```

const {
  register,
  handleSubmit,
  formState: { errors }
} = useForm();

```

We initialize React Hook Form and get the required tools.

Step 2

```

{...register("email", {
  required: "Email is required"

```

```
}}}
```

This connects the email input with React Hook Form.

It also adds validation.

Step 3

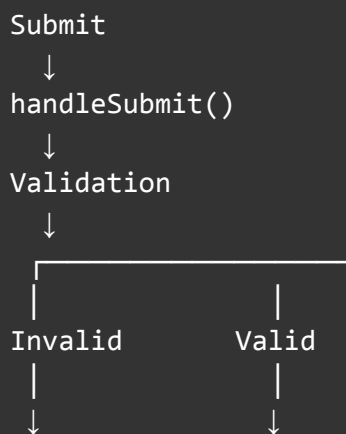
```
{errors.email && (  
  <p>{errors.email.message}</p>  
)}
```

If email validation fails, the error message is displayed.

Step 4

```
<form onSubmit={handleSubmit(onSubmit)}>
```

When the user submits:




```
errors      onSubmit()  
            ↓  
            data
```

12. Example 2 — Registration Form

Now let's create a slightly more practical registration form.

We will validate:

- Name
 - Email
 - Password
 - Age
-

Complete Example 2

```
import { useForm } from "react-hook-form";  
  
function RegisterForm() {  
  
  const {  
    register,  
    handleSubmit,  
    formState: { errors }  
  } = useForm();  
  
  const onSubmit = (data) => {  
    console.log("Registration Data:", data);  
  };  
}
```

```

return (
  <form onSubmit={handleSubmit(onSubmit)}>

    {/* Name */}

    <div>
      <label>Name</label>

      <input
        {...register("name", {
          required: "Name is required",
          minLength: {
            value: 3,
            message: "Name must be at least 3 characters"
          }
        })}
      />

      {errors.name && (
        <p>{errors.name.message}</p>
      )}
    </div>

    {/* Email */}

    <div>
      <label>Email</label>

      <input
        type="email"
        {...register("email", {
          required: "Email is required",
          pattern: {
            value: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
            message: "Enter a valid email"
          }
        })}
      />

      {errors.email && (
        <p>{errors.email.message}</p>
      )}
    </div>
  </form>
)

```

```
</div>
```

```
{/* Password */}
```

```
<div>
```

```
  <label>Password</label>
```

```
  <input
```

```
    type="password"
```

```
    {...register("password", {
```

```
      required: "Password is required",
```

```
      minLength: {
```

```
        value: 8,
```

```
        message: "Password must be at least 8 characters"
```

```
      }
```

```
    })}
```

```
  />
```

```
  {errors.password && (
```

```
    <p>{errors.password.message}</p>
```

```
  )}
```

```
</div>
```

```
{/* Age */}
```

```
<div>
```

```
  <label>Age</label>
```

```
  <input
```

```
    type="number"
```

```
    {...register("age", {
```

```
      required: "Age is required",
```

```
      min: {
```

```
        value: 18,
```

```
        message: "Age must be at least 18"
```

```
      },
```

```
      max: {
```

```
        value: 60,
```

```
        message: "Age cannot be more than 60"
```

```
      }
```

```
    })}
```

```
    />

    {errors.age && (
      <p>{errors.age.message}</p>
    )}
  </div>

  <button type="submit">
    Register
  </button>

</form>
);
}

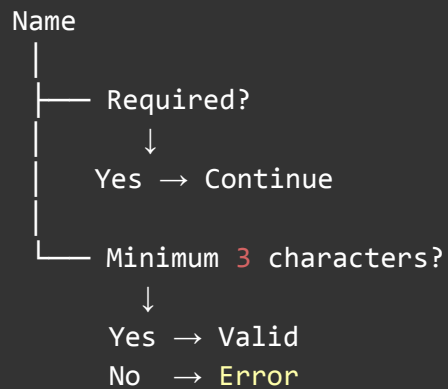
export default RegisterForm;
```

13. Example 2 — Step-by-Step Explanation

Name Validation

```
register("name", {
  required: "Name is required",
  minLength: {
    value: 3,
    message: "Name must be at least 3 characters"
  }
})
```

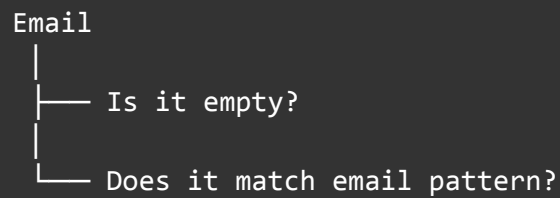
There are two validation rules:



Email Validation

```
register("email", {
  required: "Email is required",
  pattern: {
    value: /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/,
    message: "Enter a valid email"
  }
})
```

Two checks happen:



If either validation fails:

`errors.email`

will contain the error information.

Password Validation

```
register("password", {  
  required: "Password is required",  
  minLength: {  
    value: 8,  
    message: "Password must be at least 8 characters"  
  }  
})
```

The password must:

Not be empty

+

At least 8 characters

Age Validation

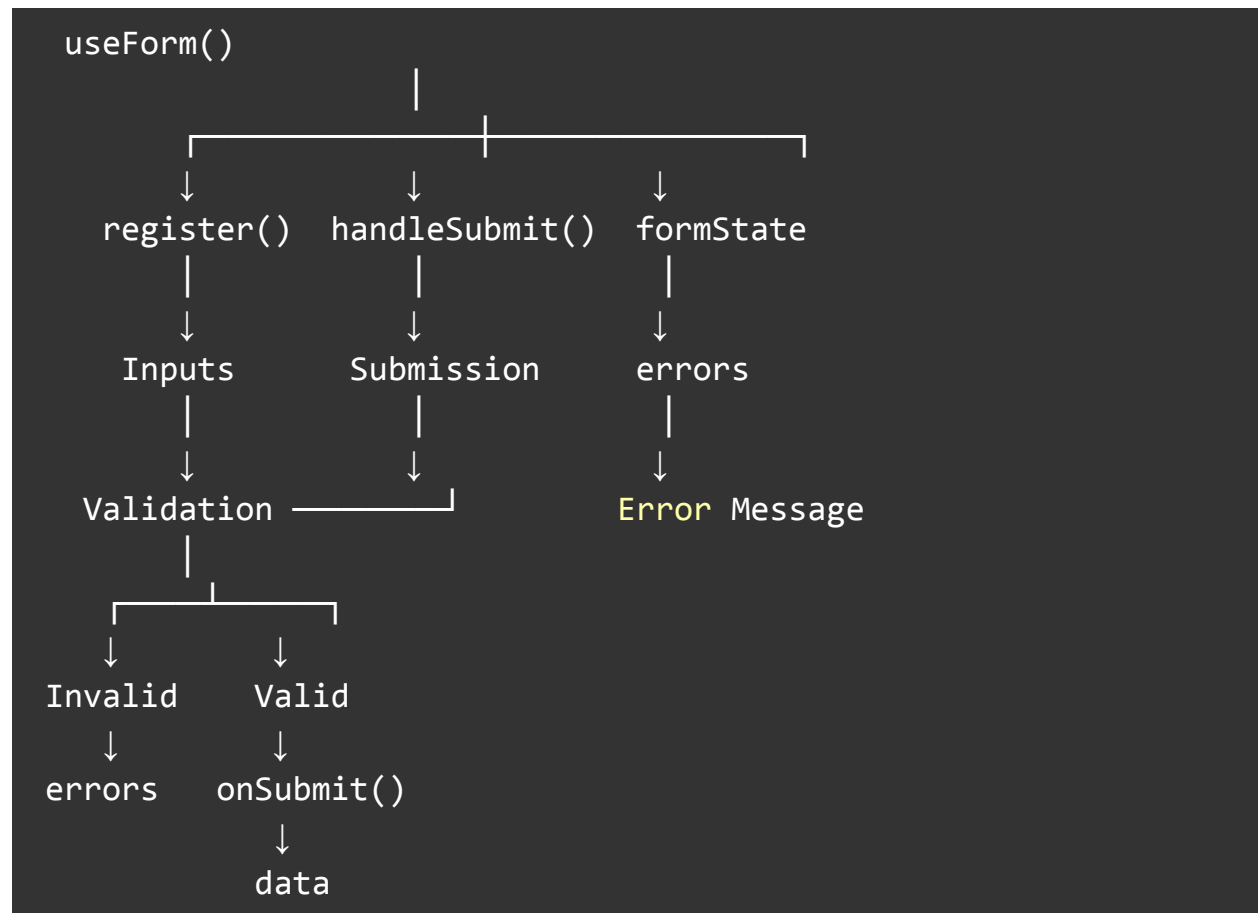
```
register("age", {  
  required: "Age is required",  
  min: {  
    value: 18,  
    message: "Age must be at least 18"  
  },  
  max: {  
    value: 60,  
    message: "Age cannot be more than 60"  
  }  
})
```

The valid range is:

$18 \leq \text{Age} \leq 60$

14. Understanding the Complete React Hook Form Flow

This is the most important flow to remember.



15. Interview Questions

Q1. What is React Hook Form?

Answer:

React Hook Form is a React library used to manage forms, form values, validation, errors, and submission with minimal code.

Q2. What does `register()` do?

Answer:

`register()` connects an input field to React Hook Form and allows the library to track its value and validation.

Q3. What is `formState`?

Answer:

`formState` contains information about the current state of the form, such as errors, submission state, and validity.

Q4. How do you access validation errors?

```
const {  
  formState: { errors }  
} = useForm();
```

Then:

`errors.email`

Q5. What does handleSubmit() do?

Answer:

`handleSubmit()` processes form submission and runs validation before calling the submit callback when the form is valid.

Q6. How do you make an input required?

```
<input
  {...register("username", {
    required: "Username is required"
  })}
/>
```